



Fourier-feature induced physics informed randomized neural network method to solve the biharmonic equation

Xi'an Li ^{a,b}, Jinran Wu ^c, Yujia Huang ^c, Zhe Ding ^{d,*}, Xin Tai ^{a,*}, Liang Liu ^a, You-Gan Wang ^e

^a Ceyear Technologies Co., Ltd, Qingdao 266555, China

^b Shandong University, Qingdao 266237, China

^c The University of Queensland, St Lucia 4067, Australia

^d Beijing University of Posts and Telecommunications, Beijing 100876, China

^e Guangdong University of Finance and Economics, Guangzhou 510320, China

ARTICLE INFO

MSC:

35J58

65N35

68T07

Keywords:

FPISRNN

Activation function

Boundary

Non-unitized domain

Linear system

ABSTRACT

To address the sensitivity of parameters and limited precision for randomized neural network method (RNN) with common activation functions, such as sigmoid, tangent, and Gaussian, in solving high-order partial differential equations (PDEs) relevant to scientific computation and engineering applications, this work develops a Fourier-feature induced physics informed single layer RNN (FPISRNN) method. This approach aims to approximate the solution for a class of fourth-order biharmonic equation with two boundary conditions on both unitized and non-unitized domains. By carefully calculating the differential and boundary operators of the biharmonic equation on discretized collections, the solution for this high-order equation is reformulated as a linear least squares minimization problem. We further evaluate the FPISRNN method with varying hidden nodes and scaling factors for uniform distribution initialization, and then determine the optimal range for these two hyperparameters. Numerical experiments and comparative analyses demonstrate that the proposed FPISRNN method is more stable, robust, precise, and efficient than other FPISRNN approaches in solving biharmonic equations for both regular and irregular domains.

1. Introduction

Partial differential equations (PDEs), which have their roots in science and engineering, have been instrumental in advancing the fields of physics, chemistry, biology, image processing, and more. In general, the analytical solutions of PDEs are often elusive, necessitating the reliance on numerical methods and approximation techniques for their resolution. This paper focuses on the numerical solution of the following biharmonic equation:

$$\Delta^2 u(\mathbf{x}) = f(\mathbf{x}), \text{ in } \Omega, \quad (1)$$

where Ω is a polygonal or polyhedral domain of dimension d in Euclidean space, with a piecewise Lipschitz boundary satisfying an interior cone condition. Here, $f(\mathbf{x}) \in L^2(\Omega)$ is a prescribed function, and Δ denotes the standard Laplace operator. The expression

* Corresponding authors.

E-mail addresses: lixian9131@163.com (X. Li), jinran.wu@uq.edu.au (J. Wu), rainfamilyh@gmail.com (Y. Huang), zhedingqd@126.com (Z. Ding), taixin@ceyear.com (X. Tai), liuliang@ceyear.com (L. Liu), ygwanguq2012@gmail.com (Y.-G. Wang).

<https://doi.org/10.1016/j.cam.2025.116635>

Received 8 November 2024; Received in revised form 23 February 2025

Available online 14 March 2025

0377-0427/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

of the biharmonic operator $\Delta^2 u(\mathbf{x})$ is given by:

$$\Delta^2 u(\mathbf{x}) = \sum_{i=1}^d \frac{\partial^4 u}{\partial x_i^4} + \sum_{i=1}^d \sum_{j=1, j \neq i}^d \frac{\partial^4 u}{\partial x_i^2 \partial x_j^2}.$$

In practice, the solution of Eq. (1) can be uniquely and precisely determined by the one of the following two boundary conditions:

- Dirichlet boundary conditions:

$$u(\mathbf{x}) = g(\mathbf{x}) \quad \text{and} \quad \frac{\partial u(\mathbf{x})}{\partial \vec{n}} = h(\mathbf{x}) \quad \text{on } \partial\Omega; \quad (2)$$

- Navier boundary conditions:

$$u(\mathbf{x}) = g(\mathbf{x}) \quad \text{and} \quad \Delta u(\mathbf{x}) = k(\mathbf{x}) \quad \text{on } \partial\Omega, \quad (3)$$

where $\partial\Omega$ denotes the boundary of Ω , and $\vec{n} = (n_1, n_2, \dots, n_d)$ is the outward normal vector on $\partial\Omega$. The functions g , h , and k describe the boundary behavior of the solution and are assumed to be sufficiently smooth.

The biharmonic equation, a high-order PDE, frequently occurs in fields such as elasticity and fluid mechanics. Its application ranges from scattered data fitting with thin plate splines [1] to the simulation of fluid mechanics [2,3] and linear elasticity [4,5]. Solving it is challenging due to the fourth-order derivative terms, particularly in complex geometries. Various traditional numerical methods for solving (1) can be broadly classified into direct (uncoupled) methods and coupled (splitting, mixed) methods.

In the direct approach, methodologies include finite difference methods (FDM) [6–9], finite volume techniques [10–12], and finite element methods (FEM) based on the variational formulation, such as non-conforming FEM [13–15] and conforming FEM [16,17]. In the coupled approach, the biharmonic equation is decomposed into two interrelated Poisson equations, solvable by methods like FDM, FEM, and mixed element methods [18–22]. Additional methods include the collocation method [23–25] and the radial basis functions (RBF) method [26–28].

Conventional methods, however, encounter challenges with irregular domains and high dimensions, often referred to as the curse of dimensionality. Machine learning, especially artificial neural networks (ANNs), has shown promise in approximating the solutions of PDEs, potentially overcoming the limitations of traditional numerical techniques. Various ANN approaches, such as randomized neural networks (RNN) and deep neural networks (DNN), have been developed to solve PDEs, including (1). Physics-informed neural network methods have also been proposed, minimizing loss and incorporating residuals and boundary conditions [29–31]. Despite their potential, these methods may suffer from limited accuracy, uncertain convergence, and slow training. Recently, a physics-informed extreme learning machines (PIELM) approach was introduced, combining physical laws with ELM to address various linear or nonlinear PDEs [32–37], it is a special type of RNN that can be traced back to the original design of perceptron proposed by Frank Rosenblatt in 1958 [38]. After that, a series of corresponding works about the perceptron are developed, such as the Random Vector Functional Link Networks and Random Weights Neural Networks presented by Pao and Schmidt [39,40], and the Echo State Networks [41]. Besides, the corresponding studies of principle and mechanism are given, such as the study of characteristics [42], the theory analysis [43], and the selection of hyperparameters [44], those works enriched the group of RNN for solving different types PDE [45–47]. RNN framework and its variants are simple neural network that the internal parameters are randomly assigned to links between hidden layers and then fixed during training, they have the merits including mesh-free operation, rapid learning, and parallel architecture compatibility, while effectively handling complex PDEs. The choice of activation function (or called transfer function) is critical for RNN methods. In terms of the regression and classification, the sigmoid $1/(1+e^{-x})$ and Gaussian e^{-x^2} functions are the general option [48–50]. For solving PDEs, experiments have shown that sigmoid, Gaussian, and hyperbolic tangent functions are ideal, producing high-precision solutions [32,34,51,52]. However, these functions may impede information transmission in high-order problems with large-scale input, as their high-order derivatives are complex. Fourier-type functions, such as $\sin(x)$, have been suggested as alternative to mitigate these issues [53–55]. Moreover, initializing RNN's internal weights and biases uniformly within $[-\delta, \delta]$ with $\delta > 0$ has been shown to enhance performance [56–58].

This work addresses the biharmonic equation (1) with Dirichlet and Navier boundary conditions by combining the Fourier spectral theorem with physics informed single RNN (PISRNN) method, resulting in a Fourier-feature induced PISRNN (FPISRNN) method. Various hidden node configurations and uniform initialization scale factors are tested to optimize hyperparameters. We further compare the performance of the FPISRNN method against that of the PISRNN methods with sigmoid, tangent, and Gaussian activation functions. Numerical results indicate that the FPISRNN method achieves exceptional performance and robustness for solving (1) on both unitized and non-unitized domains.

The paper is organized as follows: Section 2 introduces the classical single layer RNN framework. Section 3 presents the FPISRNN algorithm construction. Numerical examples evaluating FPISRNN's accuracy and efficiency are shown in Section 4, and conclusions are drawn in Section 5.

2. Preliminary for single layer RNN

This section provides a concise overview of the essential concepts and the mathematical formulation for the single layer RNN (SLRNN) model. SLRNN falls within the category of fully connected neural networks in artificial neural networks and is designed to establish a mapping $\mathcal{L}: \mathbb{R}^d \rightarrow \mathbb{R}^k$. A key characteristic of RNN is the random assignment of weights and biases in the hidden layer,

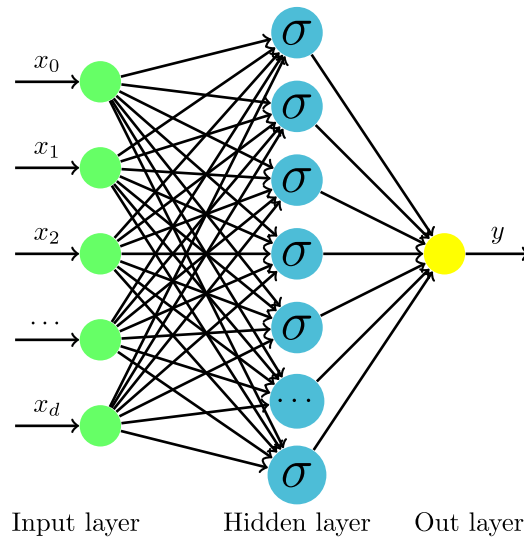


Fig. 1. Basic structure of the SLRNN.

while the output weights are determined analytically [39,50]. Mathematically, the SLRNN mapping $\ell(\mathbf{x})$ with input $\mathbf{x} \in \mathbb{R}^d$ and output $\mathbf{y} \in \mathbb{R}^k$ is defined as:

$$\mathbf{y} = (y_1, y_2, \dots, y_k) \text{ with } y_m = \sum_{i=1}^n \beta_{mi} \sigma \left(\sum_{j=1}^d w_{ij} x_j + b_i \right). \quad (4)$$

For convenience, we denote $\mathbf{W} = (\mathbf{W}^{[1]}, \mathbf{W}^{[2]}, \dots, \mathbf{W}^{[n]})^T \in \mathbb{R}^{n \times d}$, where $\mathbf{W}^{[i]} = (w_{i1}, w_{i2}, \dots, w_{id})^T \in \mathbb{R}^d$ represents the weight of the i th hidden unit in SLRNN, and $\mathbf{b} = (b_1, b_2, \dots, b_n)^T \in \mathbb{R}^n$ is the corresponding bias vector. The activation function $\sigma(\cdot)$ operates element-wise. The matrix $\beta = (\beta^{[1]}, \beta^{[2]}, \dots, \beta^{[k]})^T \in \mathbb{R}^{k \times n}$, where $\beta^{[l]} = (\beta_{l1}, \beta_{l2}, \dots, \beta_{ln})^T \in \mathbb{R}^n$, represents the weights connecting the hidden units to the outputs. A schematic of the SLRNN with n hidden units is shown in Fig. 1.

Incorporating physical laws into the RNN model introduces a novel approach called PISRNN. The PISRNN optimizes its loss function, which integrates information from governing terms as well as enforced boundary or initial conditions, by analytically calculating the weights of the outer layer. Unlike PINN, which are commonly used for solving PDEs, the PISRNN addresses several limitations associated with PINN, such as restricted accuracy, uncertain convergence, and tremendous computational demand that can lead to slow training speeds. Similar to previous RNN applications in classification and fitting tasks, the PISRNN is data-adaptive, fast convergent, and outperforms the gradient descent-based methods. Additionally, PISRNN preserves the core advantages of RNN as a universal approximator for any continuous function. Barron and Jones, as demonstrated in [59], established that a substantial class of functions can be approximated with high accuracy by finite sums of K adjustable basis functions.

Remark 1 (Function Approximation). Supposed ψ is a nonlinear function, w_k and $b_k (k = 1, 2, 3, \dots, K)$ are the prescribed parameters, then we have

$$\hat{f} = \sum_{k=1}^K c_k \psi(w_k x + b_k)$$

in which, c_k is the weights of basis function ψ . When ψ is a cosine function, the functions $\hat{f}$ with absolutely integrable Fourier transforms could be approximated by K basis functions under the L_2 error $O(1/\sqrt{K})$.

In addition, Ali Rahimi et al. expanded upon this conclusion and its applications, and demonstrated that weighted sums of random functions are dense within a Reproducing Kernel Hilbert Space (RKHS) defined by the selection of basis functions and the sampling distribution. When the number of data points is substantial, these weighted sums provide a more parsimonious functional representation that significantly accelerates computation compared to fitting the RKHS representation [55].

3. Fourier-feature induced PISRNN architecture for biharmonic equation

In this section, the unified architecture of FPISRNN is proposed to solve the biharmonic equation (1) with boundaries (2) or (3) by embracing the vanilla PISRNN method with Fourier theorem.

3.1. Vanilla PISRNN framework to solve biharmonic equation

This section provides a detailed introduction to the PISRNN method for solving the biharmonic equation (1) with boundaries (2) or (3). To obtain an ideal solution to the biharmonic equation, one can seek the solution in trial function space spanned by the SLRNN model with a high-order smooth activation function. Then, an ansatz expressed by the SLRNN is designed to approximate the solution of (1), it is:

$$u(\mathbf{x}; \mathbf{W}, \mathbf{b}) = \sigma(\mathbf{x}^T \cdot \mathbf{W}^T + \mathbf{b}^T) \cdot \beta.$$

Substituting $u(\mathbf{x}; \mathbf{W}, \mathbf{b})$ into the equation of (1) for u , we can obtain the following equation

$$\Delta^2 \left(\sum_{i=1}^N \beta_i \sigma(V_i(\mathbf{x})) \right) = f(\mathbf{x}) \quad \text{for } \mathbf{x} \in \Omega, \quad (5)$$

with $V_i(\mathbf{x}) = \mathbf{x}^T \cdot \mathbf{W}^{[i]} + b_i$ being the linear output for the i_{th} hidden unit of SLRNN and N is the number of hidden unit. Additionally, $u(\mathbf{x}; \mathbf{W}, \mathbf{b})$ still holds on the prescribed boundaries (2) or (3), we then have the following formulation

$$\begin{cases} \sum_{i=1}^N \beta_i \sigma(V_i(\mathbf{x})) = g(\mathbf{x}), \\ \partial \left(\sum_{i=1}^N \beta_i \sigma(V_i(\mathbf{x})) \right) / \partial \vec{n} = h(\mathbf{x}), \end{cases} \quad \text{for } \mathbf{x} \in \partial\Omega, \quad (6)$$

or

$$\begin{cases} \sum_{i=1}^N \beta_i \sigma(V_i(\mathbf{x})) = g(\mathbf{x}), \\ \Delta \left(\sum_{i=1}^N \beta_i \sigma(V_i(\mathbf{x})) \right) = k(\mathbf{x}), \end{cases} \quad \text{for } \mathbf{x} \in \partial\Omega. \quad (7)$$

For solving the biharmonic equation by the aforementioned PISRNN method, a collection of collocation points is sampled from both the interior and boundaries of the domain. At these collocation points, the governed function (5) and the (6) or (7) exhibit coerciveness. Let Q denote the number of random collocation points in the interior of Ω and denote by $S_I = \{\mathbf{x}_I^q\}_{q=1}^Q$, and P denote the number of random collocation points on hyperface of $\partial\Omega$ and denote by $S_B = \{\mathbf{x}_B^p\}_{p=1}^P$. Feeding the above collocation points into the pipeline of PISRNN, we have the following residual terms for Dirichlet boundaries:

$$\begin{cases} \mathcal{R}_u(\mathbf{x}_I^q) = \Delta^2 \left(\sum_{i=1}^N \beta_i \sigma(V_i(\mathbf{x}_I^q)) \right) - f(\mathbf{x}_I^q), \\ \mathcal{L}_g(\mathbf{x}_B^p) = \sum_{i=1}^N \beta_i \sigma(V_i(\mathbf{x}_B^p)) - h(\mathbf{x}_B^p) \\ \mathcal{L}_h(\mathbf{x}_B^p) = \partial \left(\sum_{i=1}^N \beta_i \sigma(V_i(\mathbf{x}_B^p)) \right) / \partial \vec{n} - h(\mathbf{x}_B^p) \end{cases} \quad (8)$$

and the last term in (8) will be replaced by

$$\mathcal{L}_k(\mathbf{x}_B^p) = \Delta \left(\sum_{i=1}^N \beta_i \sigma(V_i(\mathbf{x}_B^p)) \right) - k(\mathbf{x}_B^p), \quad (9)$$

for Navier boundaries.

To approximate the functions u well, we need to adjust the output weights of the PISRNN model to make each residual of (8) to near-zero or minimize the error of the following system of linear equations:

$$\mathbf{H}\beta = \mathbf{S} \iff \begin{bmatrix} \mathbf{H}_f \\ \mathbf{H}_g \\ \mathbf{H}_h \end{bmatrix} \beta = \begin{bmatrix} \mathbf{S}_f \\ \mathbf{S}_g \\ \mathbf{S}_h \end{bmatrix} \quad \text{or} \quad \begin{bmatrix} \mathbf{H}_f \\ \mathbf{H}_g \\ \mathbf{H}_k \end{bmatrix} \beta = \begin{bmatrix} \mathbf{S}_f \\ \mathbf{S}_g \\ \mathbf{S}_k \end{bmatrix}. \quad (10)$$

Here $\mathbf{H}_f = \mathbf{H}_B + \sum_{i=1}^d \sum_{j=1, j \neq i}^d \mathbf{H}_{ij}$ with

$$\mathbf{H}_B = \begin{bmatrix} \sigma''''(V_1(\mathbf{x}_I^1)) & \sigma''''(V_2(\mathbf{x}_I^1)) & \cdots & \sigma''''(V_N(\mathbf{x}_I^1)) \\ \sigma''''(V_1(\mathbf{x}_I^2)) & \sigma''''(V_2(\mathbf{x}_I^2)) & \cdots & \sigma''''(V_N(\mathbf{x}_I^2)) \\ \vdots & \vdots & \vdots & \vdots \\ \sigma''''(V_1(\mathbf{x}_I^Q)) & \sigma''''(V_2(\mathbf{x}_I^Q)) & \cdots & \sigma''''(V_N(\mathbf{x}_I^Q)) \end{bmatrix} \odot \left[\sum_{i=1}^d w_{i1}^4 \quad \sum_{i=1}^d w_{i2}^4 \quad \cdots \quad \sum_{i=1}^d w_{iN}^4 \right]$$

and

$$\mathbf{H}_{ij} = \begin{bmatrix} \sigma''''(V_1(\mathbf{x}_I^1)) & \sigma''''(V_2(\mathbf{x}_I^1)) & \cdots & \sigma''''(V_N(\mathbf{x}_I^1)) \\ \sigma''''(V_1(\mathbf{x}_I^2)) & \sigma''''(V_2(\mathbf{x}_I^2)) & \cdots & \sigma''''(V_N(\mathbf{x}_I^2)) \\ \vdots & \vdots & \vdots & \vdots \\ \sigma''''(V_1(\mathbf{x}_I^Q)) & \sigma''''(V_2(\mathbf{x}_I^Q)) & \cdots & \sigma''''(V_N(\mathbf{x}_I^Q)) \end{bmatrix} \odot \begin{bmatrix} w_{j1}^2 * w_{i1}^2 & w_{j2}^2 * w_{i2}^2 & \cdots & w_{jN}^2 * w_{iN}^2 \end{bmatrix}.$$

Additionally, the matrices $\mathbf{H}_g$, $\mathbf{H}_h$ and $\mathbf{H}_k$ are given by

$$\mathbf{H}_g = \begin{bmatrix} \sigma(V_1(\mathbf{x}_B^1)) & \sigma(V_2(\mathbf{x}_B^1)) & \cdots & \sigma(V_N(\mathbf{x}_B^1)) \\ \sigma(V_1(\mathbf{x}_B^2)) & \sigma(V_2(\mathbf{x}_B^2)) & \cdots & \sigma(V_N(\mathbf{x}_B^2)) \\ \vdots & \vdots & \vdots & \vdots \\ \sigma(V_1(\mathbf{x}_B^P)) & \sigma(V_2(\mathbf{x}_B^P)) & \cdots & \sigma(V_N(\mathbf{x}_B^P)) \end{bmatrix},$$

$\mathbf{H}_h = \mathbf{H}_1 \cdot n_1 + \mathbf{H}_2 \cdot n_2 + \cdots + \mathbf{H}_d \cdot n_d$ with

$$\mathbf{H}_i = \begin{bmatrix} \sigma'(V_1(\mathbf{x}_B^1)) & \sigma'(V_2(\mathbf{x}_B^1)) & \cdots & \sigma'(V_N(\mathbf{x}_B^1)) \\ \sigma'(V_1(\mathbf{x}_B^2)) & \sigma'(V_2(\mathbf{x}_B^2)) & \cdots & \sigma'(V_N(\mathbf{x}_B^2)) \\ \vdots & \vdots & \vdots & \vdots \\ \sigma'(V_1(\mathbf{x}_B^P)) & \sigma'(V_2(\mathbf{x}_B^P)) & \cdots & \sigma'(V_N(\mathbf{x}_B^P)) \end{bmatrix} \odot \begin{bmatrix} w_{i1} & w_{i2} & \cdots & w_{iN} \end{bmatrix}, \quad i = 1, 2, \dots, d,$$

and

$$\mathbf{H}_k = \begin{bmatrix} \sigma''(V_1(\mathbf{x}_B^1)) & \sigma''(V_2(\mathbf{x}_B^1)) & \cdots & \sigma''(V_N(\mathbf{x}_B^1)) \\ \sigma''(V_1(\mathbf{x}_B^2)) & \sigma''(V_2(\mathbf{x}_B^2)) & \cdots & \sigma''(V_N(\mathbf{x}_B^2)) \\ \vdots & \vdots & \vdots & \vdots \\ \sigma''(V_1(\mathbf{x}_B^P)) & \sigma''(V_2(\mathbf{x}_B^P)) & \cdots & \sigma''(V_N(\mathbf{x}_B^P)) \end{bmatrix} \odot \begin{bmatrix} \sum_{i=1}^d w_{i1}^2 & \sum_{i=1}^d w_{i2}^2 & \cdots & \sum_{i=1}^d w_{iN}^2 \end{bmatrix}.$$

In the above expressions, σ'' and σ'''' are the second and fourth-order derivatives of activation function σ , respectively, and $\odot$ stands for the Hadamard product operator. In addition, $\mathbf{S}_f = (f(\mathbf{x}_I^1), f(\mathbf{x}_I^2), \dots, f(\mathbf{x}_I^Q))^T$, $\mathbf{S}_g = (g(\mathbf{x}_B^1), g(\mathbf{x}_B^2), \dots, g(\mathbf{x}_B^P))^T$, $\mathbf{S}_h = (h(\mathbf{x}_B^1), h(\mathbf{x}_B^2), \dots, h(\mathbf{x}_B^P))^T$ and $\mathbf{S}_k = (k(\mathbf{x}_B^1), k(\mathbf{x}_B^2), \dots, k(\mathbf{x}_B^P))^T$.

When the weights and biases of the hidden layer for the PISRNN model are randomly initialized and fixed, the matrix $\mathbf{H}$ is determined for given input data, then the output weights β of PISRNN model are obtained by solving the linear system (10). The linear system $\mathbf{H}\beta = \mathbf{S}$ is solvable and meets the following several cases:

- If the coefficient matrix $\mathbf{H}$ is square and invertible, then

$$\beta = \mathbf{H}^{-1}\mathbf{S}. \quad (11)$$

- If the coefficient matrix $\mathbf{H}$ is rectangular and full rank, then

$$\beta = \mathbf{H}^\dagger \mathbf{S}, \quad (12)$$

where $\mathbf{H}^\dagger$ is the pseudo inverse of $\mathbf{H}$ defined by $\mathbf{H}^\dagger = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T$.

- If the coefficient matrix $\mathbf{H}$ is singular, then the Tikhonov regularization [60] can be utilized to overcome the ill-posedness of (10), and the β is given by

$$\beta = (\mathbf{H}^T \mathbf{H} + \lambda \mathbf{I})^{-1} \mathbf{H}^T \mathbf{S}, \quad (13)$$

where $\mathbf{I}$ is the identity matrix, and $\lambda > 0$ is the ridge parameter.

Remark 2. Generally, the coefficient matrices $\mathbf{H}$, $\mathbf{H}^T \mathbf{H}$ and $\mathbf{H}^T \mathbf{H} + \lambda \mathbf{I}$ in Eqs. (11)–(13) are dense and large-scale, then it is tough to deal with them. We can solve those linear systems by direct approaches, such as LU decomposition [61], QR decomposition [62] and SVD decomposition [63], or the iteration methods, such as GMRES [64] and multi-grid [65]. In particular, one can choose the regularization parameter λ in (13) according to the max-eigenvalue $\sigma_{\max}$ of $\mathbf{H}^T \mathbf{H}$, since the $\sigma_{\max}$ is easy to obtain. Heuristically, the λ in (13) is approximated to the 1×10^{-6} to 1×10^{-3} times $\sigma_{\max}$, it is decided by the noise level. Finally, the tolerance threshold is usually set as 1×10^{-12} for double precision.

According to the above discussions, the detailed procedures involved in the PISRNN algorithm to address the biharmonic equation (1) within finite-dimensional spaces are described in the following.

Algorithm 1 PISRNN algorithm for solving biharmonic equation (1).

- (1) Generating the points set S^k includes interior points $S_I = \{\mathbf{x}_I^q\}_{q=1}^Q$ with $\mathbf{x}_I^q \in \mathbb{R}^d$ and boundary points $S_B = \{\mathbf{x}_B^p\}_{p=1}^P$ with $\mathbf{x}_B^p \in \mathbb{R}^d$. Here, we draw the random points $\mathbf{x}_I^q$ and $\mathbf{x}_B^p$ from $\mathbb{R}^d$ with positive probability density ν , such as uniform distribution.
- (2) Selecting a shallow neural network and randomly initialize weights and biases of the hidden layer, then fixing the parameters of the hidden layer.
- (3) Embedding the input data into the implicit feature space by a linear transformation, then activating nonlinearly the results and casting them into the output layer.
- (4) Combining linearly the all output features of the hidden layer and output this results.
- (5) Computing the derivation about the input variable for the output of the PISRNN model and constructing the linear system (10).
- (6) Learning the output layer weights by the least-squares method.

3.2. The option of activation function

In terms of function approximation, the hidden nodes governed by a given activation function in the SLRNN model can be considered a set of basis functions, and then the SLRNN model produces its output through a linear aggregation of these fundamental functions. Similar to other artificial neural network (ANN) architectures, the choice of activation function is critical for the SLRNN model's performance. Since the biharmonic equation (1) studied here is a high-order PDE, an activation functions with strong regularity is preferred. Previous studies, including Fabiani et al. [51], Calabrò et al. [52], and Ma et al. [66], have investigated the influence of sigmoidal functions and radial basis functions in the SLRNN model for solving both linear and nonlinear PDEs. These studies indicate that SLRNN models with sigmoid, Gaussian, and tangent activation functions achieve commendable numerical accuracy.

Remark 3 (Lipschitz Continuity). If a function σ is continuous (i.e., $\sigma \in C^1$) and satisfies the following boundedness condition:

$$|\sigma(x)| < 1 \quad \text{and} \quad |\sigma'(x)| < 1$$

for any $x \in \mathbb{R}$. Then, we have

$$|\sigma(x) - \sigma(y)| \leq |x - y| \quad \text{and} \quad |\sigma'(x) - \sigma'(y)| < |x - y|$$

for any $x, y \in \mathbb{R}$.

The sigmoid, gaussian, and tangent functions all fulfill the Lipschitz continuity and exhibit robust regularity, they ensure the boundedness of the output for hidden units in the SLRNN network. In terms of the high-order PDEs, the high-order derivate for the activation is required. Then, the first, second, and fourth-order derivatives for the sigmoidal function $s(x) = 1/(1 + e^{-x})$, Gaussian function $\sigma(x) = e^{-x^2}$ and hyperbolic tangent function $\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$ are given by

$$\begin{cases} s'(x) = s(x) - s^2(x) \\ s''(x) = s(x) - 3s^2(x) + 2s^3(x) \\ s'''(x) = s(x) - 15s^2(x) + 50s^3(x) - 60s^4(x) + 24s^5(x), \end{cases} \quad (14)$$

$$\begin{cases} (\sigma(x))' = -2xe^{-x^2} \\ (\sigma(x))'' = -2e^{-x^2} + 4x^2e^{-x^2} \\ (\sigma(x))''' = 12e^{-x^2} - 48x^2e^{-x^2} + 16x^4e^{-x^2}, \end{cases} \quad (15)$$

and

$$\begin{cases} \tanh'(x) = 1 - \tanh^2(x) \\ \tanh''(x) = -2 \tanh(x) + 2 \tanh^3(x) \\ \tanh'''(x) = (16 \tanh(x) - 24 \tanh^3(x))(1 - \tanh^2(x)), \end{cases} \quad (16)$$

respectively. Figs. 2–4 illustrate the curves of the sigmoid, Gaussian, and tangent functions and their corresponding first-order, second-order and fourth-order derivatives, respectively.

This visualization shows that the derivatives of the sigmoid, Gaussian, and tanh functions are bounded and tend to saturate for narrow-range input. However, their performance may degrade with large-range inputs. Inspired by the Fourier-based spectral method, a global basis function can generate responses, along with its derivatives, for both narrow- and large-range inputs. Furthermore, the approximations based on such basis functions can handle data across various scales. In fact, the sine function is an ideal global Fourier-type basis function (with sine and cosine being fundamentally equivalent), it inherits the essential property

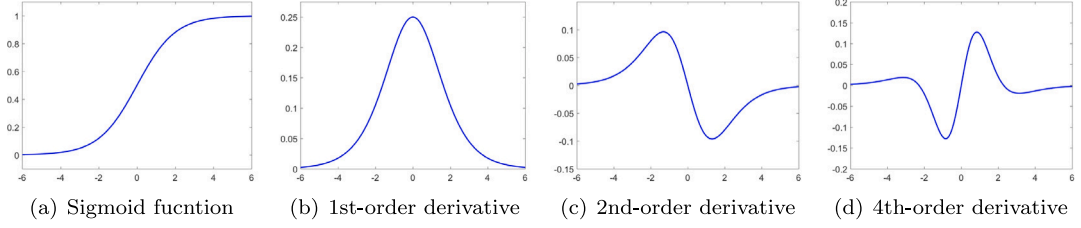


Fig. 2. The sigmoid function and its 1st-order, 2nd-order and 4th-order derivatives, respectively.

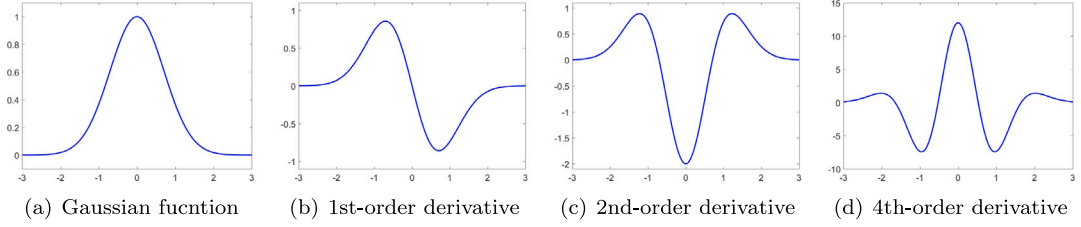


Fig. 3. The Gaussian function and its 1st-order, 2nd-order and 4th-order derivatives, respectively.

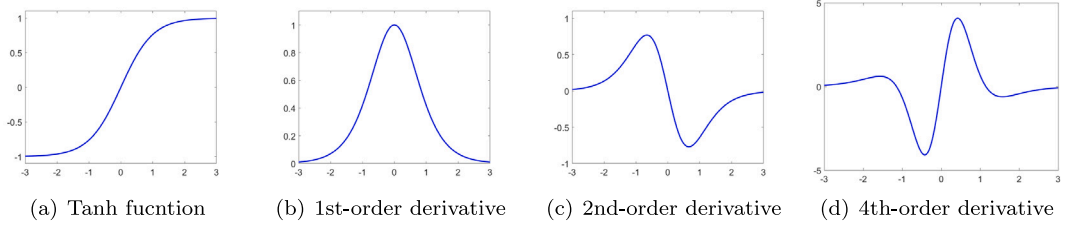


Fig. 4. The tanh function and its 1st-order, 2nd-order and 4th-order derivatives, respectively.

of randomized Fourier feature mapping and will cast the input information into different frequency boxes, then it is suitable to approximate complex target function. In this study, we adopt the sine function as the activation function in the PISRNN method, referring to this modified approach as FPISRNN.

4. Numerical experiments

4.1. Experimental setup

In this section, we apply the proposed FPISRNN method to solve the two- and three-dimensional biharmonic equation (1) in both unitized and non-unitized domains. Additionally, PISRNN models using sigmoid, Gaussian and tanh as activation functions are established as baselines for solving (1) and are referred to as SPISRNN, GPISRNN, and TPISRNN, respectively. Following the results in Dong and Yang [57], the weight matrix and bias vector for the hidden layers of each model are initialized uniformly within the interval $[-\delta, \delta]$.

A criterion is provided to evaluate the performance of the aforementioned four methods, given by

$$\text{REL} = \sqrt{\frac{\sum_{i=1}^{N'} |\tilde{u}(\mathbf{x}^i) - u^*(\mathbf{x}^i)|^2}{\sum_{i=1}^{N'} |u^*(\mathbf{x}^i)|^2}},$$

where $\tilde{u}(\mathbf{x}^i)$ and $u^*(\mathbf{x}^i)$ denote the approximate solution obtained by numerical methods and the exact solution, respectively, for the collection set $\{\mathbf{x}^i\}_{i=1}^{N'}$, with N' being the number of collection points.

All experiments are implemented in Numpy (version 1.21.0) on a workstation equipped with 256 GB RAM and a single NVIDIA GeForce RTX 4090 Ti GPU with 24 GB of memory.

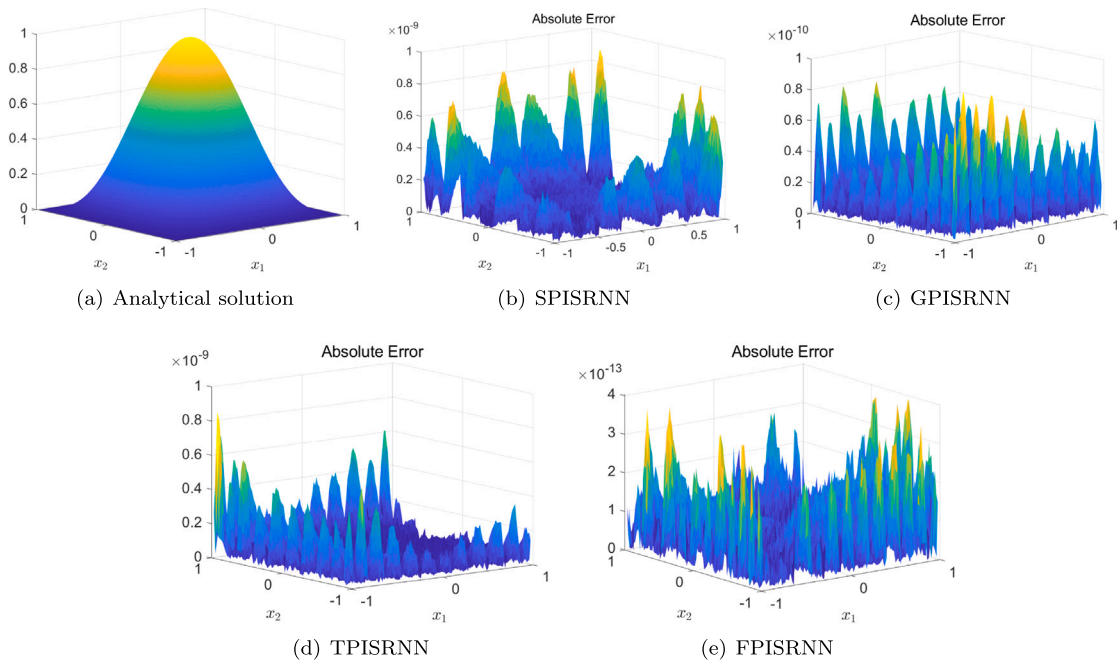


Fig. 5. Numerical results of four PISRNN methods for solving Example 4.1 on domain $[-1, 1] \times [-1, 1]$.

4.2. Numerical examples for Dirichlet boundary

Example 4.1. To start, we aim to approximate the solution to the biharmonic equation (1) with Dirichlet boundary in the regular domain $\Omega = [x_1^{\min}, x_1^{\max}] \times [x_2^{\min}, x_2^{\max}]$. An exact solution is given by

$$u(x_1, x_2) = [(x_1 - x_1^{\min})(x_1^{\max} - x_1)]^2 [(x_2 - x_2^{\min})(x_2^{\max} - x_2)]^2$$

such that

$$\begin{aligned} f(x_1, x_2) = & 24(x_2 - x_2^{\min})^2(x_2^{\max} - x_2)^2 + 24(x_1 - x_1^{\min})^2(x_1^{\max} - x_1)^2 \\ & + 2[2(x_1^{\min} + x_1^{\max} - 2x_1)^2 - 4(x_1 - x_1^{\min})(x_1^{\max} - x_1)] \\ & \times [2(x_2^{\min} + x_2^{\max} - 2x_2)^2 - 4(x_2 - x_2^{\min})(x_2^{\max} - x_2)]. \end{aligned}$$

The boundary functions $g(x_1, x_2)$ and $h(x_1, x_2)$ can be obtained easily by direct computation.

First, we approximate the solution of (1) using the four methods on 16,384 equidistant grid points distributed across regular domains: $[-1, 1] \times [-1, 1]$, $[0, 5] \times [0, 5]$, $[-4, 6] \times [-3, 7]$, and $[5, 15] \times [0, 10]$. In addition to the grid points, the training set includes 10,000 random points within Ω and 4000 random points on $\partial\Omega$. The number of hidden nodes for each of the four PISRNN methods is set to 1000.

Fig. 5 shows the point-wise absolute error of the four PISRNN methods on the domain $[-1, 1] \times [-1, 1]$. Table 1 presents the optimal value of δ , the REL, and the runtime for each PISRNN method across the different domains.

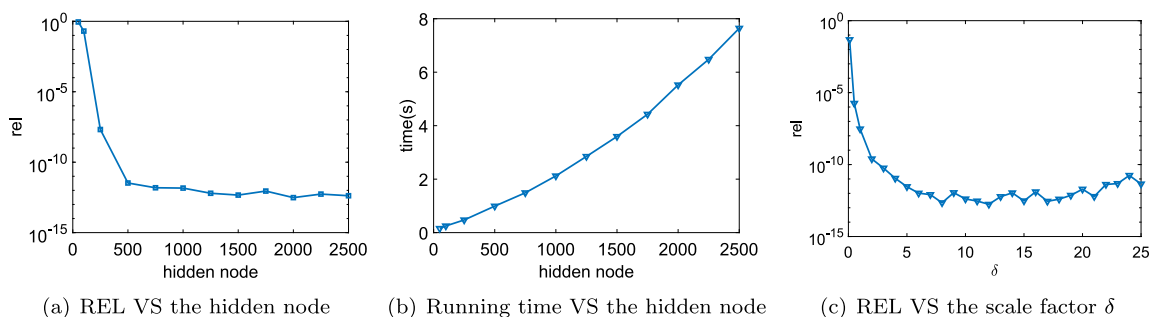
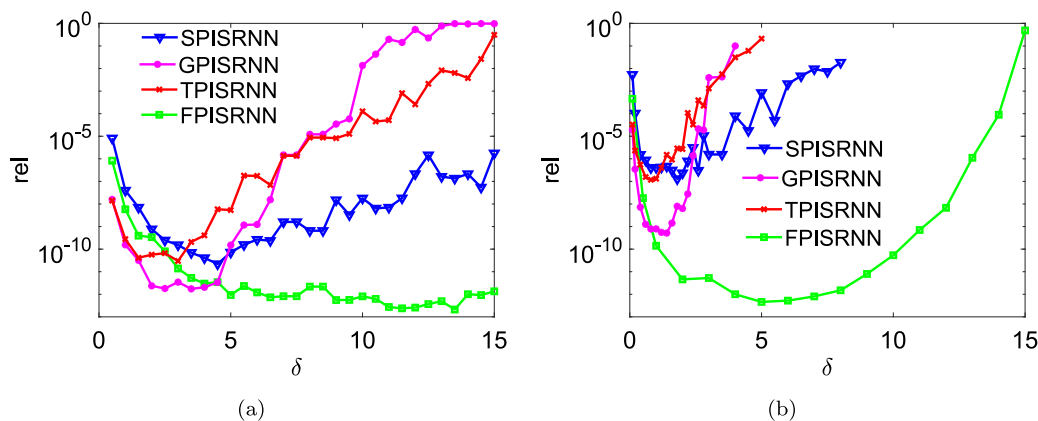
Next, we examine the impact of the scale factor δ and the number of hidden nodes on the FPISRNN model in the domain $[-1, 1] \times [-1, 1]$, with results illustrated in Fig. 6. Finally, comparison curves for REL as a function of the scale factor δ for the four PISRNN methods for the domains $[-1, 1] \times [-1, 1]$ and $[0, 5] \times [0, 5]$ are shown in Fig. 7.

Based on the point-wise absolute error distribution shown in Figs. 5(b)–5(e) and the data in Table 1, it can be concluded that the FPISRNN method outperforms other types of PISRNN method. As observed in the curves in Fig. 6(a), the relative error does not decrease with an increasing number of hidden units, indicating that the performance of the FPISRNN method stabilizes as the number of hidden neurons grows. The time-neuron curve in Fig. 6(b) demonstrates a superlinear increase in runtime with the addition of hidden neurons, though this increase is gradual. Furthermore, Fig. 6(c) illustrates that the FPISRNN model remains stable and robust across variations in the scale factor σ on the domain $[-1, 1] \times [-1, 1]$. The REL curves in Fig. 7 confirm that FPISRNN maintains stability and allows for a broader range of scale factor δ values across both unitized and non-unitized domains. In summary, our FPISRNN method not only achieves high accuracy but also demonstrates efficiency and robustness. Additionally, it is noteworthy that the coefficient matrix in the final linear system is full rank, given that the number of samples exceeds the number of hidden nodes.

For comparison, we employ the finite difference method (FDM) with a five-point central difference scheme to solve the coupled scheme of (1) on the domain $[0, 5] \times [0, 5]$. The FDM is implemented in MATLAB (version 12.0.0.341360, R2019a). For the FPISRNN

Table 1REL, δ and runtime for four PISRNN methods to Example 4.1 on various domain.

Domain		SPISRNN	GPISRNN	TPISRNN	FPISRNN
$[-1, 1] \times [-1, 1]$	δ	6.0	1.5	1.4	8.0
	REL	4.799×10^{-10}	4.059×10^{-11}	1.396×10^{-10}	2.223×10^{-13}
	Time (s)	8.203	3.939	3.265	2.875
$[0, 5] \times [0, 5]$	δ	2.0	0.8	1.4	5.0
	REL	3.358×10^{-7}	5.795×10^{-10}	1.819×10^{-7}	2.573×10^{-13}
	Time (s)	6.676	3.081	3.129	2.152
$[-4, 6] \times [-3, 7]$	δ	0.5	0.3	0.2	1.0
	REL	4.029×10^{-7}	2.436×10^{-8}	2.955×10^{-7}	4.935×10^{-11}
	Time (s)	8.077	4.046	3.328	3.031
$[5, 15] \times [0, 10]$	δ	0.1	0.2	0.12	0.8
	REL	3.503×10^{-3}	1.187×10^{-4}	9.331×10^{-4}	6.981×10^{-8}
	Time (s)	8.138	3.891	3.251	2.843

**Fig. 6.** REL VS. the hidden node (left), running time VS. the hidden node (middle) and REL VS. the scale factor δ (right) for FPISRNN method to Example 4.1 on domain $[-1, 1] \times [-1, 1]$.**Fig. 7.** REL VS. the scale factor δ for the four PISRNN models to solve Example 4.1 on regular domain $[-1, 1] \times [-1, 1]$ (left) and $[0, 5] \times [0, 5]$ (right).**Table 2**REL and runtime of FDM and FPISRNN for Example 4.1 on $[0, 5] \times [0, 5]$.

	Number of mesh grid	64×64	128×128	256×256	512×512	1024×1024
FDM	REL	1.65×10^{-3}	4.14×10^{-4}	1.04×10^{-4}	2.59×10^{-5}	6.47×10^{-6}
	Time (s)	0.028	0.037	0.127	0.485	2.05
FPISRNN	REL	4.20×10^{-13}	2.57×10^{-13}	3.42×10^{-13}	1.64×10^{-13}	2.43×10^{-13}
	Time (s)	1.47	2.09	4.71	16.67	69.16

model, the number of hidden nodes is set to 1000, and the scale factor δ for initializing weights and biases is set to 5.0. The numerical results in Table 2 indicate that the proposed FPISRNN method maintains stability across various mesh grid sizes and significantly outperforms FDM in terms of accuracy, particularly on sparse grids.

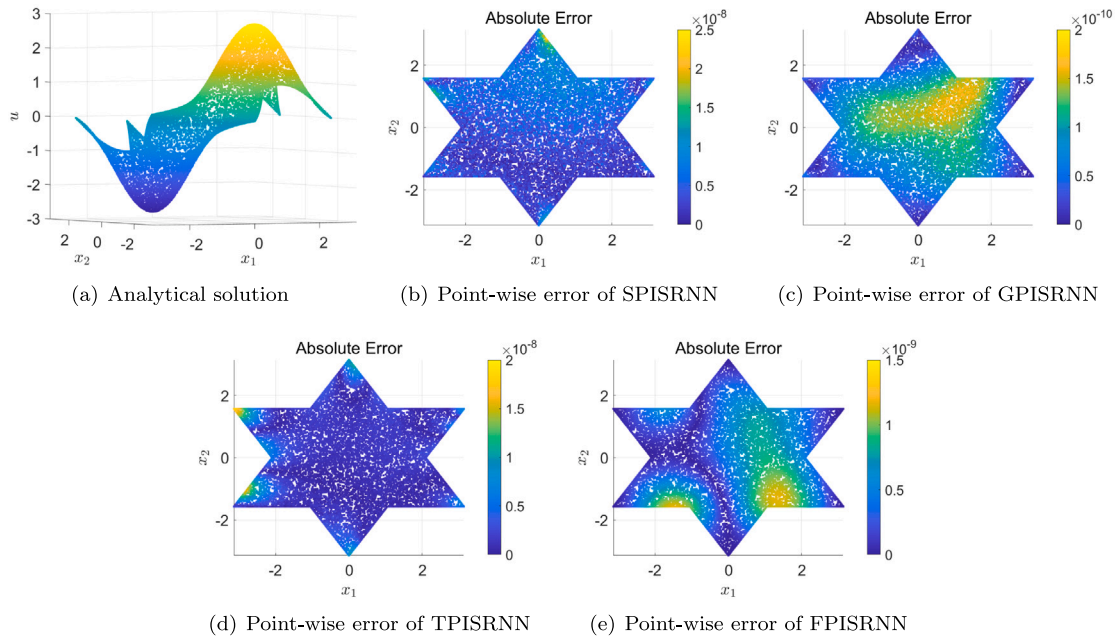


Fig. 8. Numerical results for four PISRNN methods to Example 4.2 on hexagram domain derived from $[-\pi, \pi] \times [-\pi, \pi]$.

Table 3

REL, δ and runtime for four PISRNN methods to Example 4.2.

Domain		SPISRNN	GPISRNN	TPISRNN	FPISRNN
$[-\pi, \pi] \times [-\pi, \pi]$	δ	1.2	1.4	0.6	8.5
	REL	3.469×10^{-9}	6.949×10^{-11}	1.453×10^{-9}	3.876×10^{-10}
	Time (s)	6.121	2.794	2.868	2.095
$[0, 3\pi] \times [\pi, 2\pi]$	δ	0.6	0.6	0.3	8.5
	REL	0.792	0.0147	0.422	7.866×10^{-5}
	Time (s)	4.879	2.541	2.651	2.011

Example 4.2. We consider the biharmonic equation (1) with Dirichlet boundary conditions in two-dimensional space and obtain its numerical solution within two hexagram-shaped domains, Ω , derived from $[-\pi, \pi] \times [-\pi, \pi]$ and $[0, 3\pi] \times [\pi, 2\pi]$, respectively. An analytical solution is given by

$$u(x_1, x_2) = \sin(x_1)e^{\cos(x_2)},$$

such that

$$f(x_1, x_2) = \sin(x_1)e^{\cos(x_2)} - 2\sin(x_1)e^{\cos(x_2)}[\sin^2(x_2) - e^{\cos(x_2)}] \\ + \sin(x_1)e^{\cos(x_2)}[\sin^4(x_2) - 6\sin^2(x_2)\cos(x_2) + 3\cos^2(x_2) - 4\sin^2(x_2) + \cos(x_2)].$$

The boundary constraint functions $g(x_1, x_2)$ and $h(x_1, x_2)$ on $\partial\Omega$ can be obtained by direct computation, but are omitted here for brevity.

In this example, the setup for the four PISRNN methods is identical to that in Example 4.1. An approximate solution to (1) is derived by applying the four methods on 19,214 collection points located within the hexagram domains Ω . Numerical results are presented in Table 3 for the two scenarios, and the point-wise absolute error for the four PISRNN methods is shown in Fig. 8 over the hexagram domain derived from $[-\pi, \pi] \times [-\pi, \pi]$.

Additionally, Fig. 9 illustrates the variation in relative error and runtime concerning the number of hidden nodes, as well as the variation in relative error with the scale factor δ for the FPISRNN on the hexagram domain derived from $[-\pi, \pi] \times [-\pi, \pi]$. Finally, Fig. 10 plots the REL comparison curves for the four PISRNN methods, showing how REL varies with the scale factor δ across the hexagram domains derived from $[-\pi, \pi] \times [-\pi, \pi]$ and $[0, 3\pi] \times [\pi, 2\pi]$.

Based on the data in Table 3 and the point-wise absolute errors depicted in Figs. 8(b)–8(e), we can infer that the FPISRNN method outperforms the PISRNN methods with sigmoid, tanh, and Gaussian activation functions. As observed in the curves in Fig. 16(a), the performance of the FPISRNN method stabilizes as the number of hidden nodes increases. The time-neuron curve in Fig. 16(b) shows that the runtime exhibits a superlinear increase with the growing number of hidden nodes, although the increase remains

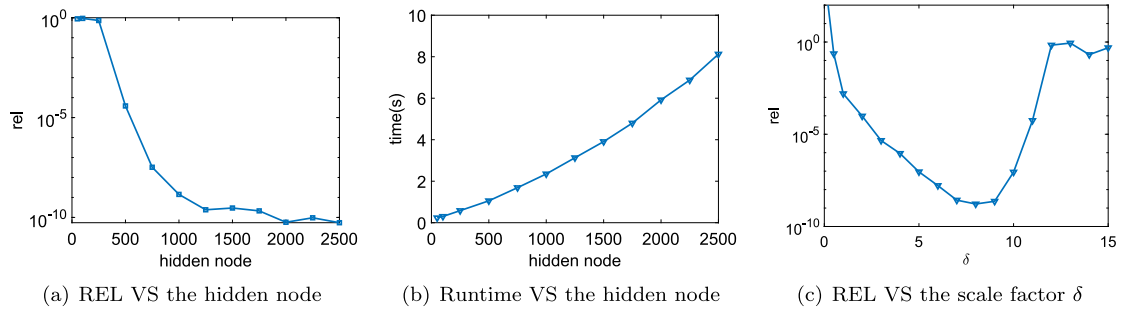


Fig. 9. REL VS. the hidden node (left), runtime VS. the hidden node (middle) and REL VS. the scale factor δ (right) to FPISRNN method to Example 4.2 on hexagram domain derived from $[-\pi, \pi] \times [-\pi, \pi]$.

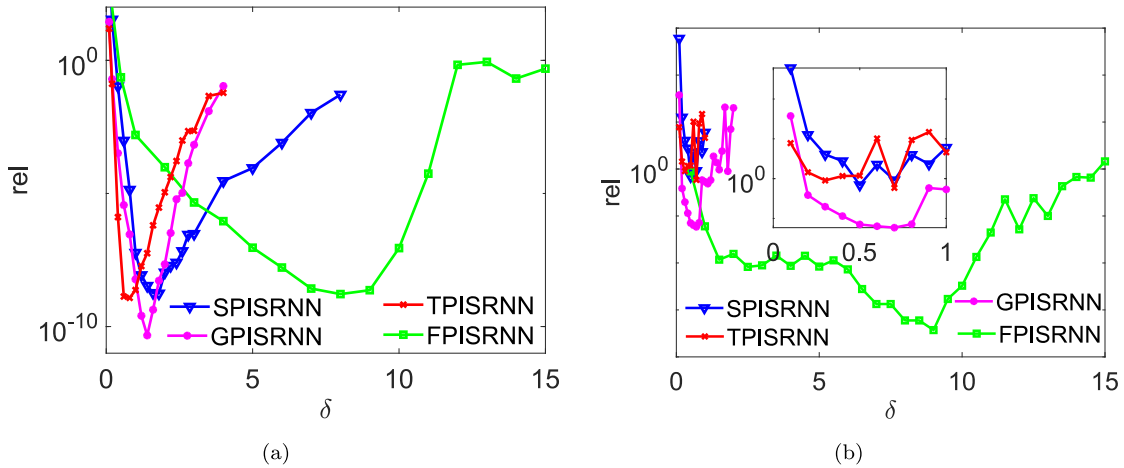


Fig. 10. REL VS. the scale factor δ for four PISRNN methods to Example 4.1 on hexagram domain derived from $[-\pi, \pi] \times [-\pi, \pi]$ (left) and $[0, 3\pi] \times [\pi, 2\pi]$ (right), respectively.

gradual. Lastly, Figs. 9(c) and 10 illustrate that the FPISRNN method remains stable and robust across variations in the scale factor σ . In summary, our FPISRNN method demonstrates not only high accuracy but also efficiency and robustness.

Example 4.3. We now approximate the solution of the biharmonic equation (1) in a three-dimensional cubic domain $\Omega = [1, 3] \times [1, 3] \times [1, 3]$ containing one large hole (cyan) and eight smaller holes (red and blue), as shown in Fig. 11. An analytical solution and its corresponding source term are given by

$$u(x_1, x_2, x_3) = 50e^{-0.25(x_1+x_2+x_3)},$$

and

$$f(x_1, x_2, x_3) = \frac{225}{128}e^{-0.25(x_1+x_2+x_3)},$$

respectively. The boundary functions $g(x_1, x_2, x_3)$ and $h(x_1, x_2, x_3)$ can be directly obtained from the exact solution; however, we omit them here for brevity.

In this example, the number of hidden units for the SLRNN network in the four PISRNN methods is set to 2000. The testing set consists of 40,000 random points within the cut planes, excluding the holes, and includes planes parallel to the xoy and $yo z$ coordinate planes. Additionally, 6000 points randomly sampled from the six boundary surfaces are integrated into the testing set. The results are presented in Figs. 12–14.

Based on the point-wise errors shown in Figs. 12(b)–12(e), our proposed FPISRNN method outperforms the other three methods in solving the biharmonic equation (1) in three-dimensional space. The relative errors presented in Fig. 14 indicate that the FPISRNN method remains stable across various scale factors δ and is superior to the other methods. Furthermore, the curves of relative error and runtime in Fig. 13 demonstrate the favorable stability and robustness of the FPISRNN method as both the number of hidden nodes and the scale factor δ increase. It is also noteworthy that the coefficient matrix in the final linear system is full rank for this example, eliminating the need for adjustment through Tikhonov regularization.

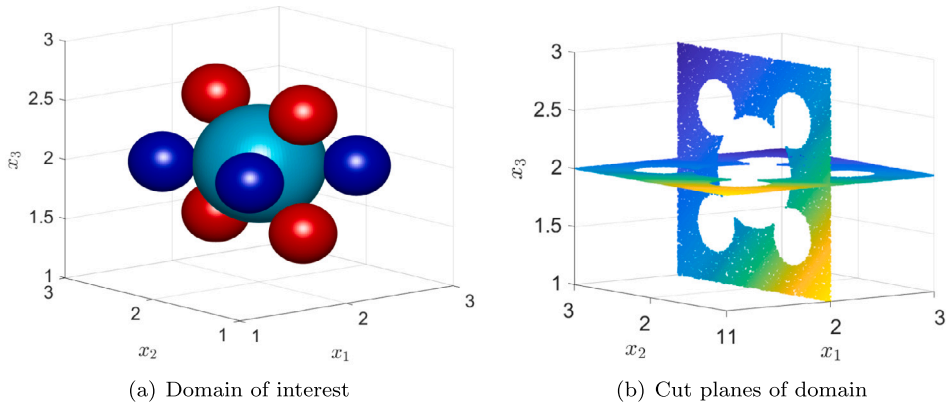


Fig. 11. The domain of interest with several holes and its cut planes.

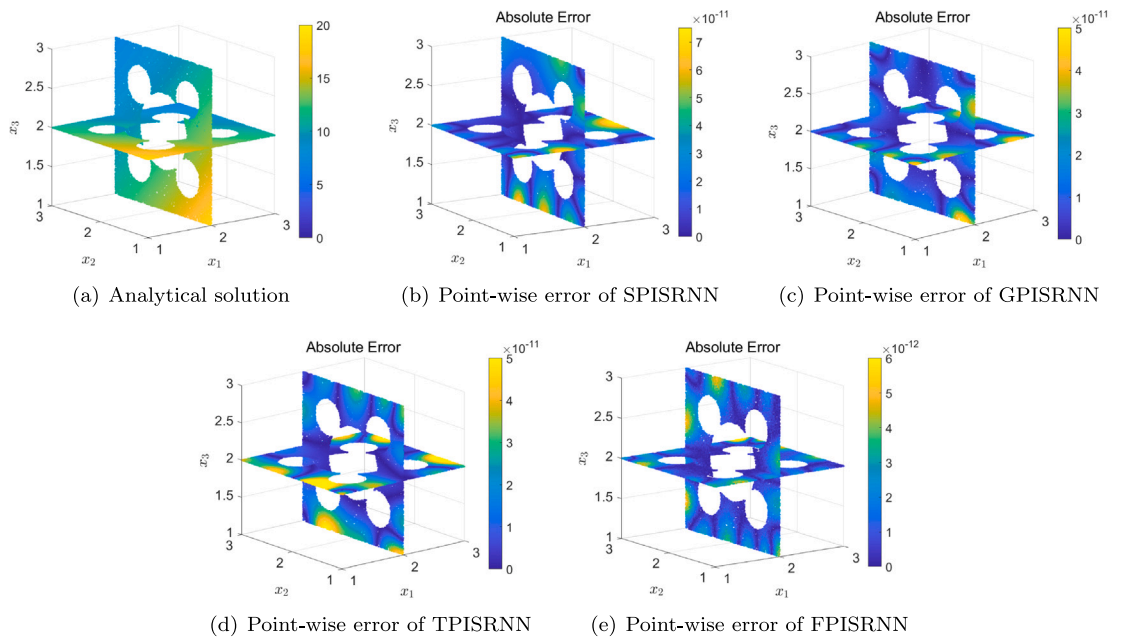


Fig. 12. Numerical results for four PISRNN methods to Example 4.3.

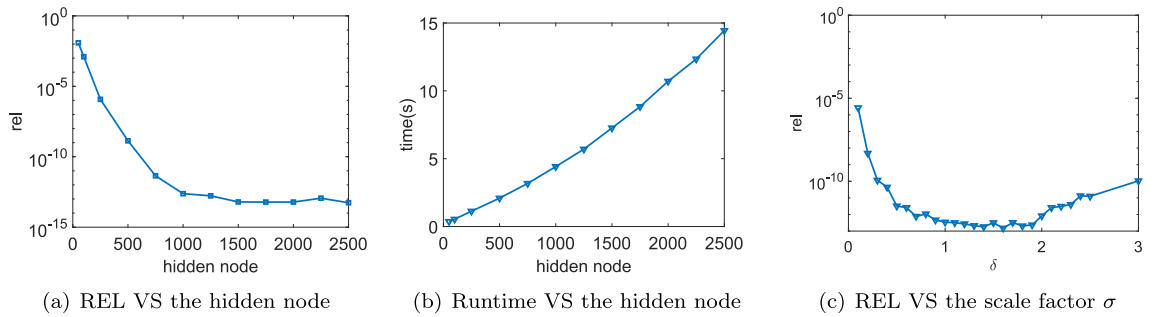


Fig. 13. REL VS. the hidden node (left), runtime VS. the hidden node (middle) and REL VS. the scale factor σ (right) for the PISRNN method to Example 4.3.

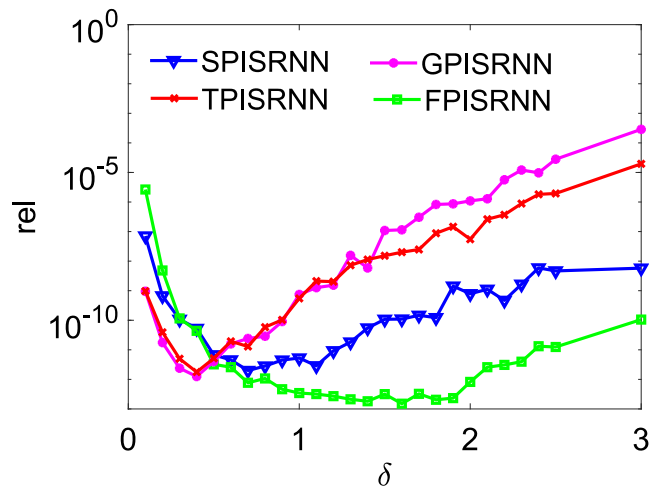


Fig. 14. REL VS. the scale factor δ for four PISRNN models to Example 4.3.

Table 4

REL, δ and runtime for four PISRNN methods to Example 4.4.

Domain		SPISRNN	GPISRNN	TPISRNN	FPISRNN
[0, 1] $\times$ [0, 1]	δ	6.5	4.1	3.2	9.0
	REL	1.222×10^{-13}	1.315×10^{-14}	1.552×10^{-13}	1.891×10^{-15}
	Time (s)	11.958	4.648	4.823	2.775
[0, 4] $\times$ [0, 4]	δ	3.0	1.5	4.0	11.0
	REL	2.471	0.0979	2.016	9.418×10^{-8}
	Time (s)	6.795	3.133	3.121	2.178

4.3. Numerical examples for Navier boundary

Example 4.4. In this example, our goal is to solve the biharmonic equation (1) with Navier boundary conditions in a regular rectangular domain $\Omega = [x_1^{\min}, x_1^{\max}] \times [x_2^{\min}, x_2^{\max}]$. An exact solution is given by

$$u(x_1, x_2) = \sin(x_1^2 + x_2^2),$$

which naturally leads to the essential boundary function $g(x_1, x_2)$. The function $h(x_1, x_2)$ is defined as

$$k(x_1, x_2) = 2 \cos(x_1^2 + x_2^2) - 4x_1^2 \sin(x_1^2 + x_2^2) + 2 \cos(x_1^2 + x_2^2) - 4x_2^2 \sin(x_1^2 + x_2^2).$$

With careful calculations, the expression for the force term is determined as

$$f(x_1, x_2) = 16x_1^4 \sin(x_1^2 + x_2^2) + 16x_2^4 \sin(x_1^2 + x_2^2) - 64x_1^2 \cos(x_1^2 + x_2^2) - 64x_2^2 \cos(x_1^2 + x_2^2) + 32x_1^2 x_2^2 \sin(x_1^2 + x_2^2) - 32 \sin(x_1^2 + x_2^2).$$

In this example, the setup for the four PISRNN methods is identical to that in Example 4.1. An approximate solution of (1) is obtained by applying the four methods on 16,384 grid points within the rectangular domains $[0, 1] \times [0, 1]$ and $[0, 4] \times [0, 4]$. The numerical results are listed in Table 4 for these two scenarios, and the point-wise absolute errors for the four PISRNN methods is shown in Fig. 15 on the domain $[0, 1] \times [0, 1]$.

Additionally, Fig. 16 illustrates the variation in relative error and runtime concerning the number of hidden nodes, as well as the variation in relative error with the scale factor δ for FPISRNN on the domain $[0, 1] \times [0, 1]$. Finally, Fig. 17 plots the comparison curves for REL as a function of the scale factor δ for the four PISRNN methods on the rectangular domains $[0, 1] \times [0, 1]$ and $[0, 4] \times [0, 4]$, respectively.

Based on the point-wise errors depicted in Figs. 15(b)–15(e) and the relative errors presented in Fig. 16, our proposed FPISRNN method outperforms the other three methods for solving the biharmonic equation (1) with Navier boundary conditions in both unitized and non-unitized domains. The FPISRNN method remains stable across various scale factors δ , and its runtime increases linearly with the number of hidden nodes. Additionally, the curves of relative error and runtime in Fig. 17 highlight the favorable stability and robustness of the FPISRNN method as both the number of hidden nodes and the scale factor δ increase.

Example 4.5. We consider the biharmonic equation (1) on two porous domains Ω , derived from the two-dimensional domains $[-1, 1] \times [-\pi, \pi]$ and $[0, 4] \times [0, 4\pi]$, respectively. The exact solution is given by

$$u(x_1, x_2) = e^{x_1} \sin(x_2),$$

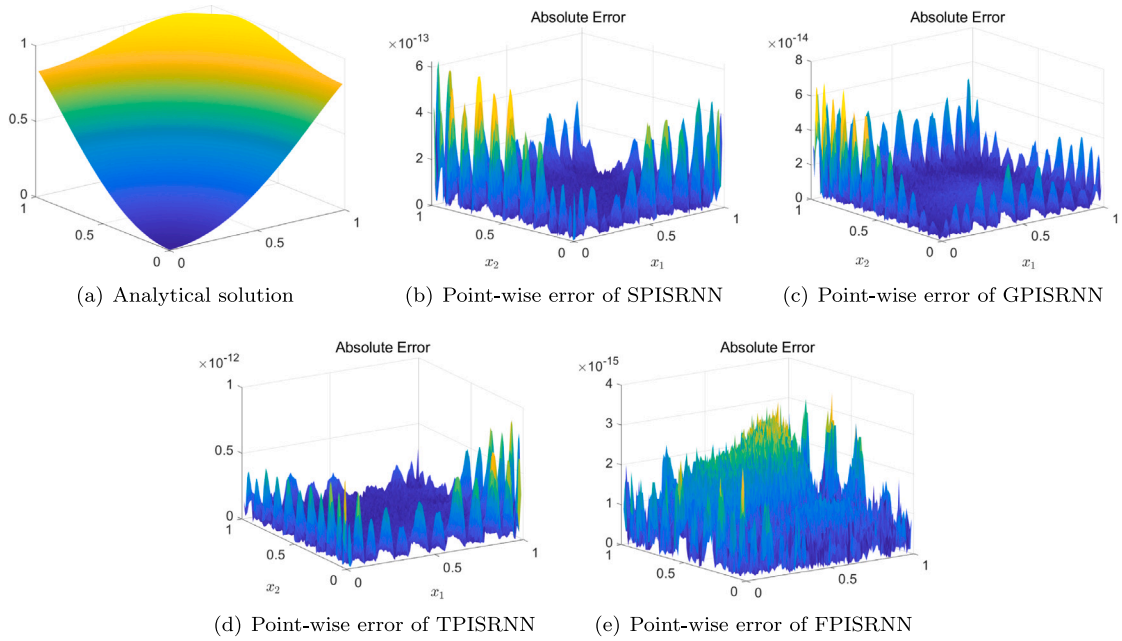


Fig. 15. Numerical results for four PISRNN methods to Example 4.4 on domain $[0, 1] \times [0, 1]$.

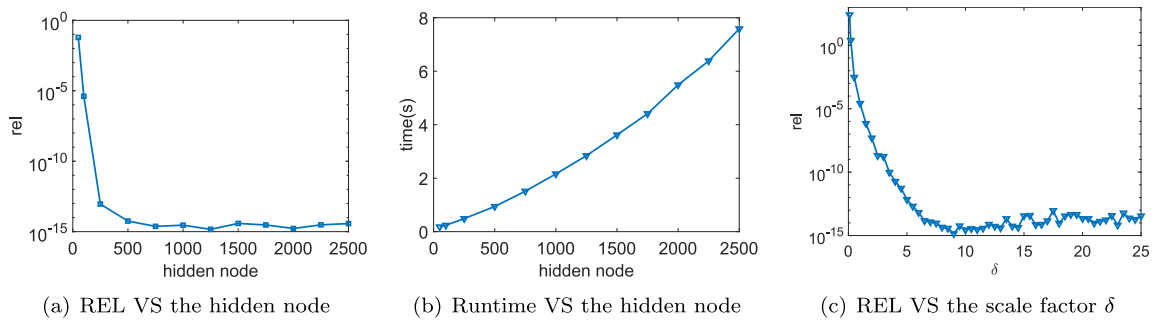


Fig. 16. REL VS. the hidden nodes (left), runtime VS. the hidden node (middle) and REL VS. the scale factor δ (right) for FPISRNN method to Example 4.4 on domain $[0, 1] \times [0, 1]$.

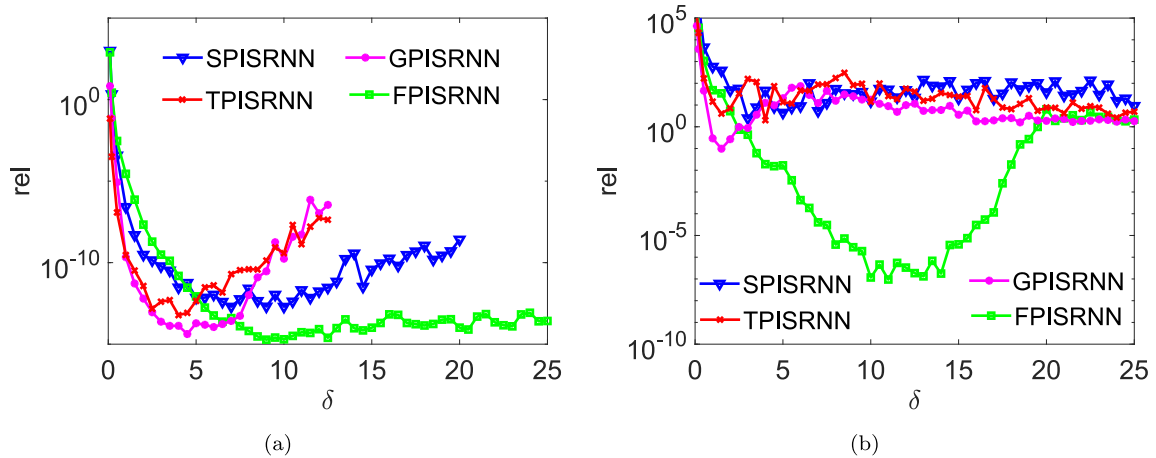


Fig. 17. REL VS. the scale factor δ for the four PISRNN methods to Example 4.4 on regular domains $[0, 1] \times [0, 1]$ (left) and $[0, 4] \times [0, 4]$ (right).

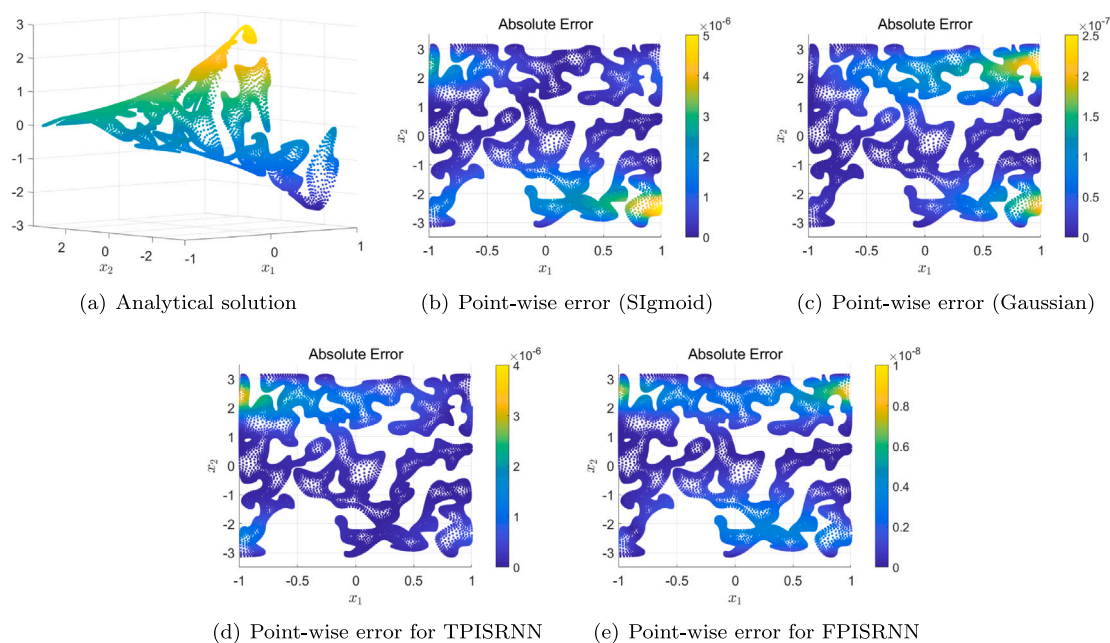


Fig. 18. Numerical results for four PISRNN methods to Example 4.5 on porous domain derived from $[-1, 1] \times [-\pi, \pi]$.

Table 5

REL, δ and runtime for four PISRNN methods to Example 4.5.

Domain		SPISRNN	GPISRNN	TPISRNN	FPISRNN
$[-1, 1] \times [-\pi, \pi]$	δ	0.7	0.4	0.4	2.5
	REL	1.013×10^{-6}	6.185×10^{-8}	6.286×10^{-7}	2.072×10^{-9}
	Time (s)	5.902	2.788	2.715	1.924
$[0, 4] \times [0, 4\pi]$	δ	0.6	0.4	0.3	1.2
	REL	1.508	0.668	1.417	5.581×10^{-4}
	Time (s)	6.095	2.748	2.842	1.941

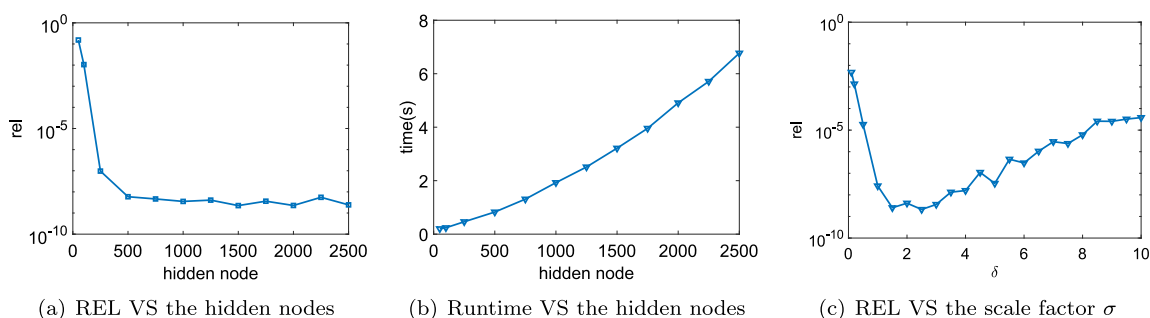


Fig. 19. REL VS. the hidden nodes (left), runtime VS. the hidden nodes (middle) and REL VS. the scale factor σ (right) to FPISRNN method for Example 4.5 on the porous domain.

such that $f(x_1, x_2) = 0$ and $k(x_1, x_2) = 0$. The boundary conditions $g(x_1, x_2)$ can be easily obtained through direct calculation.

In this example, the setup for the four PISRNN methods is identical to that in Example 4.4. An approximation of the solution to the biharmonic equation (1) is obtained using the four methods on 20,000 points randomly sampled from the hexagonal domain Ω . The numerical results are presented in Figs. 18–20 and in Table 5.

Based on the results in Fig. 18 and Table 5, the FPISRNN method demonstrates a remarkable ability to achieve a satisfactory approximation of the solution to (1) with Navier boundary conditions, even within an irregular domain in two-dimensional space.

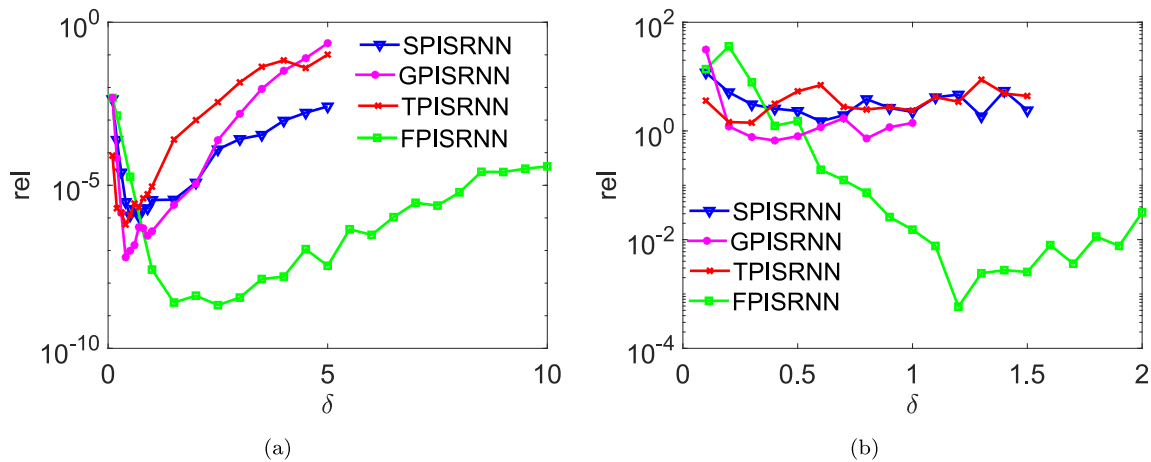


Fig. 20. REL VS. the scale factor δ to the four PISRNN models for solving Example 4.5 on porous domain from $[-1, 1] \times [-\pi, \pi]$ (left) and $[0, 4] \times [0, 4\pi]$ (right).

Its performance surpasses that of the other three methods. As shown in Figs. 19(a) and 19(b), the FPISRNN method remains stable as the number of hidden nodes varies, with computational complexity increasing only linearly as hidden nodes increase. Regarding the scale factor σ , the FPISRNN method shows insensitivity across a broad range, while the other three methods are sensitive in the unitized domain and fail to converge on the non-unitized domain. Additionally, the coefficient matrix in the final linear systems for this example is full rank.

Example 4.6. We consider the biharmonic equation (1) in three-dimensional space and obtain its numerical solution within a closed domain Ω , bounded by two spherical surfaces with radii $r_1 = 0.2$ and $r_2 = 1.0$, respectively.

An exact solution is given by

$$u(x_1, x_2, x_3) = \sin(\pi x_1) \sin(\pi x_2) \sin(\pi x_3),$$

which naturally determines the boundary conditions $g(x_1, x_2, x_3)$ on $\partial\Omega$. The second-order boundary function is given by $k(x_1, x_2, x_3) = -3\pi^2 \sin(\pi x_1) \sin(\pi x_2) \sin(\pi x_3)$. Through careful calculation, the source term is determined as $f(x_1, x_2, x_3) = 9\pi^4 \sin(\pi x_1) \sin(\pi x_2) \sin(\pi x_3)$.

In this example, we obtain the numerical solution of (1) using the FPISRNN method with 2000 hidden units in Ω . The other settings of the FPISRNN method are the same as in Example 4.4. The testing set consists of 3000 mesh grid points on the spherical surface with radius $r = 1.2$. In addition to the mesh grid points, the training set includes 40,000 randomly sampled points from the domain Ω and 20,000 random points equally sampled from the inner and outer spherical surfaces. The numerical results are shown in Figs. 21–23.

For the spherical cavity in three-dimensional space, the point-wise errors in Figs. 21(c)–21(f) indicate that our proposed FPISRNN method produces lower errors than those achieved using sigmoid, hyperbolic tangent, and Gaussian functions in solving the biharmonic equation (1). The curves for relative error and runtime in Figs. 22 and 23 demonstrate the excellent stability and robustness of the FPISRNN method across varying scale factors σ , with a notably broad effective range for the scale factor δ . Furthermore, the runtime of FPISRNN increases linearly as the number of hidden nodes gradually grows.

5. Conclusion

We have introduced a Fourier-induced physics informed single layer RNN (FPISRNN) to solve biharmonic problems, utilizing the Fourier expansion of a given function within a simple SLRNN network. Similar to other mesh-free approaches like PINN and radial basis function models, FPISRNN operates independently of a mesh and does not require an initial guess. Unlike the PINN framework, FPISRNN is an efficient solution for approximating (1) without needing parameter updates via iterative, gradient descent-based methods. Additionally, we examined the effects of different activation functions — tanh, Gaussian, sigmoid, and sine — on the performance of the PISRNN method. The computational results strongly suggest that the proposed method achieves high accuracy and efficiency in solving (1) across both regular and irregular domains.

It is also worth noting that random projection neural networks have been applied not only to forward problems in steady-state and time-dependent scenarios [56,67] but have also shown effectiveness in addressing inverse problems [45,68]. These networks outperform traditional solvers [67] and other machine learning architectures [68], excelling in both numerical accuracy and computational efficiency. Given these advantages, future work will focus on extending our neural network models to address other PDE-related problems.

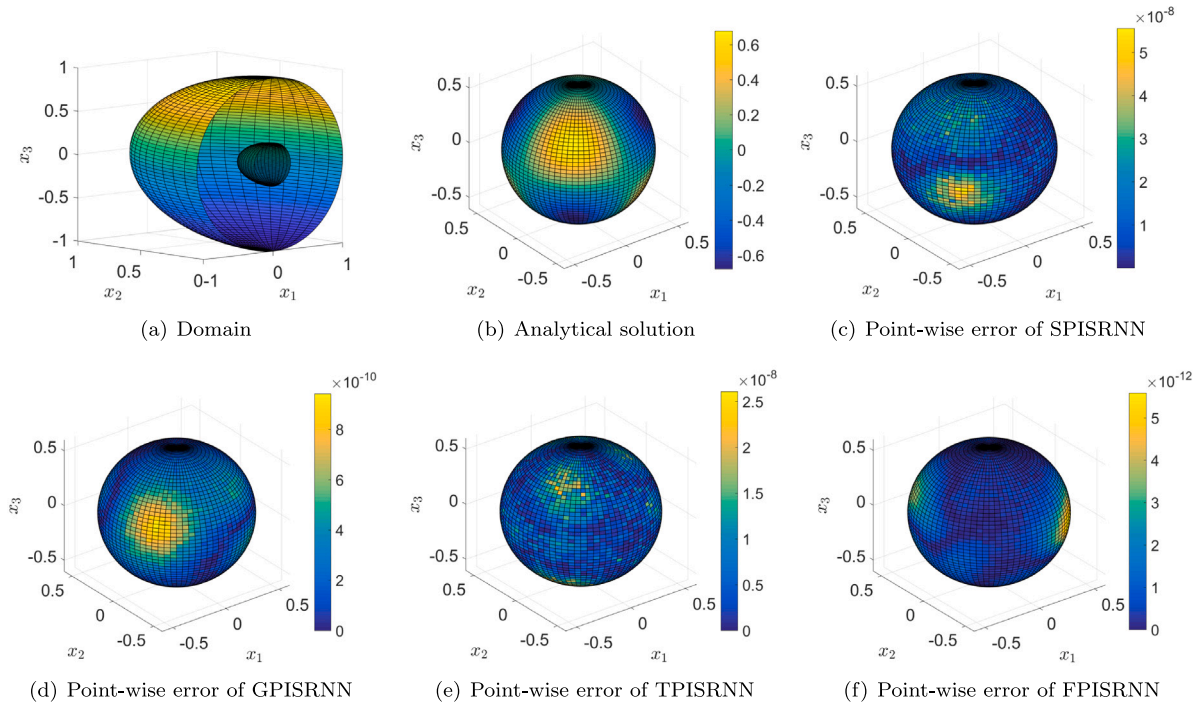


Fig. 21. Numerical results for four PISRRN methods to Example 4.3 on the cut planes.

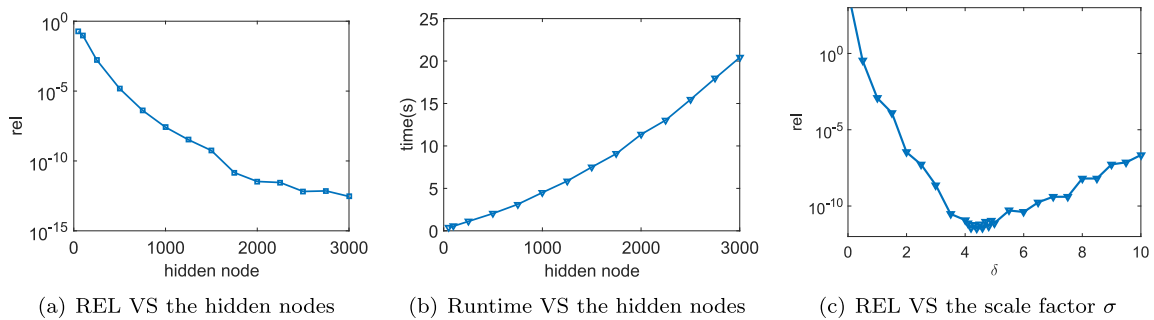


Fig. 22. REL VS. the hidden nodes (left), runtime VS. the hidden nodes (middle), and REL VS. the scale factor σ (right) to PISRRN models for Example 4.6 on the cut planes.

CRediT authorship contribution statement

Xi'an Li: Conceptualization, Methodology, Investigation, Validation, Software, Writing – original draft. **Jinran Wu:** Conceptualization, Methodology, Validation, Writing – review & editing. **Yujia Huang:** Conceptualization, Investigation, Writing – review & editing. **Zhe Ding:** Conceptualization, Writing – review & editing. **Xin Tai:** Writing – review & editing, Funding acquisition, Project administration. **Liang Liu:** Writing – review & editing, Project administration. **You-Gan Wang:** Writing – review & editing, Project administration.

Acknowledgments

This study was supported by the Special Funds of Taishan Scholars Project, China and Natural Science Foundation of Shandong Province, China (No. ZR2024QF057). The authors are grateful to the anonymous referees for their constructive suggestions and helpful comments, which greatly improved the original manuscript of this paper. All authors have read and agreed to the published version of the manuscript.

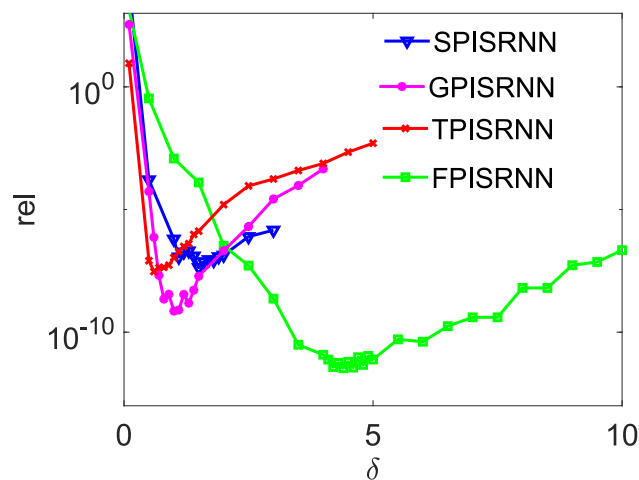


Fig. 23. REL VS. the scale factor δ to the four PISRNN models for solving Example 4.6 on sphere domain.

Data availability

Data will be made available on request.

References

- [1] G. Wahba, Spline Models for Observational Data, SIAM, 1990.
- [2] L. Greengard, M.C. Kropinski, An integral equation approach to the incompressible Navier–Stokes equations in two dimensions, SIAM J. Sci. Comput. 20 (1) (1998) 318–336.
- [3] J.H. Ferziger, M. Perić, Computational Methods for Fluid Dynamics, Springer, 2002.
- [4] S. Christiansen, Integral equations without a unique solution can be made useful for solving some plane harmonic problems, IMA J. Appl. Math. 16 (2) (1975) 143–159.
- [5] C. Constanda, The boundary integral equation method in plane elasticity, Proc. Amer. Math. Soc. 123 (11) (1995) 3385–3396.
- [6] M.M. Gupta, R.P. Manohar, Direct solution of the biharmonic equation using noncoupled approach, J. Comput. Phys. 33 (2) (1979) 236–248.
- [7] I. Altas, J. Dym, M.M. Gupta, R.P. Manohar, Multigrid solution of automatically generated high-order discretizations for the biharmonic equation, SIAM J. Sci. Comput. 19 (5) (1998) 1575–1585.
- [8] M. Ben-Artzi, J.-P. Croisille, D. Fishelov, A fast direct solver for the biharmonic problem in a rectangular grid, SIAM J. Sci. Comput. 31 (1) (2008) 303–333.
- [9] B. Bialecki, A fourth order finite difference method for the Dirichlet biharmonic problem, Numer. Algorithms 61 (2012) 351–375.
- [10] C.-J. Bi, L.-K. Li, Mortar finite volume method with adini element for biharmonic problem, J. Comput. Math. (2004) 475–488.
- [11] T. Wang, A mixed finite volume element method based on rectangular mesh for biharmonic equations, J. Comput. Appl. Math. 172 (1) (2004) 117–130.
- [12] R. Eymard, T. Gallouët, R. Herbin, A. Linke, Finite volume schemes for the biharmonic problem on general meshes, Math. Comp. 81 (280) (2012) 2019–2048.
- [13] G.A. Baker, Finite element methods for elliptic equations using nonconforming elements, Math. Comp. 31 (137) (1977) 45–59.
- [14] P. Lascaux, P. Lesaint, Some nonconforming finite elements for the plate bending problem, Rev. Fr. Autom. Inform. Rech. Opér. Anal. Numér. 9 (R1) (1975) 9–53.
- [15] L.S.D. Morley, The triangular equilibrium element in the solution of plate bending problems, Aeronaut. Q. 19 (2) (1968) 149–169.
- [16] O.C. Zienkiewicz, R.L. Taylor, P. Nithiarasu, The Finite Element Method for Fluid Dynamics, Butterworth-Heinemann, 2013.
- [17] P.G. Ciarlet, The Finite Element Method for Elliptic Problems, SIAM, 2002.
- [18] F. Brezzi, M. Fortin, Mixed and Hybrid Finite Element Methods, vol. 15, Springer Science & Business Media, 2012.
- [19] X.-l. Cheng, W. Han, H.-c. Huang, Some mixed finite element methods for biharmonic equation, J. Comput. Appl. Math. 126 (1–2) (2000) 91–109.
- [20] C. Davini, I. Pitacco, An unconstrained mixed method for the biharmonic problem, SIAM J. Numer. Anal. 38 (3) (2000) 820–836.
- [21] B.P. Lamichhane, A stabilized mixed finite element method for the biharmonic equation based on biorthogonal systems, J. Comput. Appl. Math. 235 (17) (2011) 5188–5197.
- [22] O. Stein, E. Grinspun, A. Jacobson, M. Wardetzky, A mixed finite element method with piecewise linear elements for the biharmonic equation on surfaces, 2019, arXiv preprint arXiv:1911.08029.
- [23] N. Mai-Duy, H. See, T. Tran-Cong, A spectral collocation technique based on integrated Chebyshev polynomials for biharmonic problems in irregular domains, Appl. Math. Model. 33 (1) (2009) 284–299.
- [24] B. Bialecki, A. Karageorghis, Spectral Chebyshev collocation for the Poisson and biharmonic equations, SIAM J. Sci. Comput. 32 (5) (2010) 2995–3019.
- [25] B. Bialecki, G. Fairweather, A. Karageorghis, J. Maack, A quadratic spline collocation method for the Dirichlet biharmonic problem, Numer. Algorithms 83 (1) (2020) 165–199.
- [26] N. Mai-Duy, R. Tanner, An effective high order interpolation scheme in BIEM for biharmonic boundary value problems, Eng. Anal. Bound. Elem. 29 (3) (2005) 210–223.
- [27] H. Adibi, J. Es'haghi, Numerical solution for biharmonic equation using multilevel radial basis functions and domain decomposition methods, Appl. Math. Comput. 186 (1) (2007) 246–255.
- [28] X. Li, J. Zhu, S. Zhang, A meshless method based on boundary integral equations and radial basis functions for biharmonic-type problems, Appl. Math. Model. 35 (2) (2011) 737–751.
- [29] H. Guo, X. Zhuang, T. Rabczuk, A deep collocation method for the bending analysis of Kirchhoff plate, Comput. Mater. Contin. 59 (2) (2019) 433–456.

- [30] Z. Huang, S. Chen, W. Chen, L. Peng, Neural network method for thin plate bending problem, *Chin. J. Solid Mech.* 42 (6) (2021) 10.
- [31] M. Vahab, E. Haghighat, M. Khaleghi, N. Khalili, A physics-informed neural network approach to solution and identification of biharmonic equations of elasticity, *J. Eng. Mech.* 148 (2) (2022) 04021154.
- [32] V. Dwivedi, B. Srinivasan, Physics informed extreme learning machine (pielm)—a rapid method for the numerical solution of partial differential equations, *Neurocomputing* 391 (2020) 96–118.
- [33] V. Dwivedi, B. Srinivasan, Solution of biharmonic equation in complicated geometries with physics informed extreme learning machine, *J. Comput. Inf. Sci. Eng.* 20 (6) (2020) 061004.
- [34] S. Li, G. Liu, S. Xiao, Extreme learning machine with kernels for solving elliptic partial differential equations, *Cogn. Comput.* 15 (2) (2023) 413–428.
- [35] Y. Wang, S. Dong, An extreme learning machine-based method for computational PDEs in higher dimensions, *Comput. Methods Appl. Mech. Engrg.* 418 (2024) 116578.
- [36] K. Joshi, V. Snigdha, A.K. Bhattacharya, Physics informed extreme learning machines with residual variation diminishing scheme for nonlinear problems with discontinuous surfaces, *IEEE Access* (2024).
- [37] X. Liu, W. Yao, W. Peng, W. Zhou, Bayesian physics-informed extreme learning machine for forward and inverse PDE problems with noisy data, *Neurocomputing* 549 (2023) 126425.
- [38] F. Rosenblatt, The perceptron: a probabilistic model for information storage and organization in the brain, *Psychol Rev* 65 (6) (1958) 386.
- [39] W.F. Schmidt, M.A. Kraaijveld, R.P. Duin, et al., Feed forward neural networks with random weights, in: *International Conference on Pattern Recognition*, IEEE Computer Society Press, 1992, 1–1.
- [40] Y.-H. Pao, G.-H. Park, D.J. Sobajic, Learning and generalization characteristics of the random vector functional-link net, *Neurocomputing* 6 (2) (1994) 163–180.
- [41] H. Jaeger, Adaptive nonlinear system identification with echo state networks, *Adv. Neural Inf. Process. Syst.* 15 (2002).
- [42] Y.-H. Pao, Y. Takefuji, Functional-link net computing: theory, system architecture, and functionalities, *Comput. Syst.* 25 (5) (1992) 76–79.
- [43] A. Rahimi, B. Recht, Random features for large-scale kernel machines, *Adv. Neural Inf. Process. Syst.* 20 (2007).
- [44] E.L. Bolager, I. Burak, C. Datar, Q. Sun, F. Dietrich, Sampling weights of deep neural networks, *Adv. Neural Inf. Process. Syst.* 36 (2023) 63075–63116.
- [45] S. Dong, Y. Wang, A method for computing inverse parametric PDE problems with random-weight neural networks, *J. Comput. Phys.* (2023) 112263.
- [46] H. Dang, F. Wang, Local randomized neural networks with hybridized discontinuous Petrov–Galerkin methods for Stokes–Darcy flows, *Phys. Fluids* 36 (8) (2024).
- [47] J. Sun, S. Dong, F. Wang, Local randomized neural networks with discontinuous Galerkin methods for partial differential equations, *J. Comput. Appl. Math.* 445 (2024) 115830.
- [48] Z.X. Chen, H.Y. Zhu, Y.G. Wang, A modified extreme learning machine with sigmoidal activation functions, *Neural Comput. Appl.* 22 (2013) 541–550.
- [49] S. Ding, H. Zhao, Y. Zhang, X. Xu, R. Nie, Extreme learning machine: algorithm, theory and applications, *Artif. Intell. Rev.* 44 (2015) 103–115.
- [50] B. Igel'nik, Y.-H. Pao, Stochastic choice of basis functions in adaptive function approximation and the functional-link net, *IEEE Trans. Neural Netw.* 6 (6) (1995) 1320–1329.
- [51] G. Fabiani, F. Calabrò, L. Russo, C. Siettos, Numerical solution and bifurcation analysis of nonlinear partial differential equations with extreme learning machines, *J. Sci. Comput.* 89 (2021) 1–35.
- [52] F. Calabrò, G. Fabiani, C. Siettos, Extreme learning machine collocation for the numerical solution of elliptic PDEs with sharp gradients, *Comput. Methods Appl. Mech. Engrg.* 387 (2021) 114188.
- [53] G.-B. Huang, L. Chen, C.-K. Siew, Universal approximation using incremental constructive feedforward networks with random hidden nodes, *IEEE Trans. Neural Netw.* 17 (4) (2006) 879–892.
- [54] G.-B. Huang, Z. Bai, L.L.C. Kasun, C.M. Vong, Local receptive fields based extreme learning machine, *IEEE Comput. Intell. Mag.* 10 (2) (2015) 18–29.
- [55] A. Rahimi, B. Recht, Uniform approximation of functions with random bases, in: *2008 46th Annual Allerton Conference on Communication, Control, and Computing*, IEEE, 2008, pp. 555–561.
- [56] S. Dong, Z. Li, Local extreme learning machines and domain decomposition for solving linear and nonlinear partial differential equations, *Comput. Methods Appl. Mech. Engrg.* 387 (2021) 114129.
- [57] S. Dong, J. Yang, On computing the hyperparameter of extreme learning machines: Algorithm and application to computational PDEs, and comparison with classical and high-order finite elements, *J. Comput. Phys.* 463 (2022) 111290.
- [58] S. Dong, Z. Li, A modified batch intrinsic plasticity method for pre-training the random coefficients of extreme learning machines, *J. Comput. Phys.* 445 (2021) 110585.
- [59] L.K. Jones, A simple lemma on greedy approximation in Hilbert space and convergence rates for projection pursuit regression and neural network training, *Ann. Statist.* (1992) 608–613.
- [60] J. Adriaola, On the role of tikhonov regularizations in standard optimization problems, 2022, *arXiv preprint arXiv:2207.01139*.
- [61] R.H. Bartels, G.H. Golub, The simplex method of linear programming using LU decomposition, *Commun. ACM* 12 (5) (1969) 266–268.
- [62] W. Gander, Algorithms for the QR decomposition, *Res. Rep.* 80 (02) (1980) 1251–1268.
- [63] G.W. Stewart, On the early history of the singular value decomposition, *SIAM Rev.* 35 (4) (1993) 551–566.
- [64] Y. Saad, M.H. Schultz, GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems, *SIAM J. Sci. Stat. Comput.* 7 (3) (1986) 856–869.
- [65] W. Hackbusch, *Multi-Grid Methods and Applications*, vol. 4, Springer Science & Business Media, 2013.
- [66] M. Ma, J. Yang, R. Liu, A novel structure automatic-determined Fourier extreme learning machine for generalized Black–Scholes partial differential equation, *Knowl.-Based Syst.* 238 (2022) 107904.
- [67] G. Fabiani, E. Galaris, L. Russo, C. Siettos, Parsimonious physics-informed random projection neural networks for initial value problems of ODEs and index-1 DAEs, *Chaos: Interdiscip. J. Nonlinear Sci.* 33 (4) (2023).
- [68] E. Galaris, G. Fabiani, I. Gallos, I. Kevrekidis, C. Siettos, Numerical bifurcation analysis of pdes from lattice Boltzmann model simulations: a parsimonious machine learning approach, *J. Sci. Comput.* 92 (2) (2022) 34.